Tutorial 5: Simulation, OLS Residual Properties, and Inference (Chapter 2 - C8)

Wenhui Xu

ECO375

Contents

Overview

This tutorial is a simulation-heavy script that:

1. Generates data from known distributions,
2. Runs OLS,
3. Demonstrates confidence intervals, hypothesis tests, and prediction intervals,
4. Uses Monte Carlo simulation to visualize sampling variation.

The closest match in Wooldridge (2019) is **Chapter 2, Computer Exercise C8**, which explicitly asks you to simulate data from Uniform and Normal distributions, estimate the regression, and verify key residual properties.

Computer Exercise (C8) — What the problem is asking

This exercise is a "controlled universe" simulation. You generate data where the true model is known:

$$y = 1 + 2x + u, \quad x \sim \text{Uniform}(0, 10), \quad u \sim \text{Normal}(0, 36).$$

Then you check what OLS does in one sample, and how results change across samples.

What you must produce (deliverables)

You should be able to report:

1. **Summary stats for x** (mean and standard deviation) from 500 simulated observations.
2. **Summary stats for u** (mean and standard deviation), and a sentence explaining why the sample mean is not exactly zero.
3. **OLS regression results** from regressing y on x, and a comparison of estimates to the true values (1 and 2).
4. **OLS residual identities** (two exact-in-sample properties, up to rounding).
5. A comparison showing these identities generally **do not hold** if you replace residuals with the true errors u in a finite sample.
6. A second run with a new sample showing estimates change, and an explanation (sampling variation).

Step-by-step plan

Step 1 — Simulate x and summarize (C8-i)

- Draw 500 values from Uniform(0,10).
- Compute `mean(x)` and `sd(x)`.

Step 2 — Simulate u and summarize (C8-ii)

- Draw 500 values from Normal(0,36). (Because variance is 36, the SD is 6.)
- Compute `mean(u)` and `sd(u)`.
- Explain: even if **E(u)=0**, a finite sample mean is almost never exactly 0.

Step 3 — Generate y and run OLS (C8-iii)

- Construct `y = 1 + 2*x + u`.
- Run `lm(y ~ x)`.
- Compare the estimated intercept and slope to the true values (1 and 2).
- Explain why they differ in a single sample: the estimates are random variables.

Step 4 — Verify OLS residual identities (C8-iv)

- Obtain residuals `uhat <- resid(fit)`.
- Check (up to rounding):
  - `sum(uhat) = 0`
  - `sum(x * uhat) = 0`

Step 5 — Repeat Step 4 using true errors u (C8-v)

- Compute:
  - `sum(u)`
  - `sum(x * u)`
- These are generally not zero in a finite sample.
- Interpretation: the residual identities hold **by construction** for OLS residuals, not for the unobserved errors.

Step 6 — Re-run with a new sample (C8-vi)

- Change the seed and re-generate x, u, y.
- Re-run the regression and compare coefficients.
- Explain why they change: different random draws imply different samples.

Common pitfalls (and how to avoid them)

- **Wrong Normal SD:** Normal(0,36) means variance = 36, so SD = 6 (not 36).
- **Forgetting the intercept:** the residual identities in Step 4 require an intercept in the regression (use `lm(y ~ x)`, not `lm(y ~ 0 + x)`).
- **Confusing u and uhat:** the "sum-to-zero" and "orthogonality" equalities are guaranteed for `uhat`, not for the true `u`.
- **Expecting exact equality to true coefficients:** OLS estimates will not equal (1,2) in one sample; that is the point of the exercise.
- **Seed confusion:** if you do not set the seed, your results will change every knit; set `set.seed()` for reproducibility.

Part A — A complete solution to C8 (with clean R code)

A1. Generate x and summarize (C8-i)

```r
set.seed(375)  # reproducibility

n <- 500
x <- runif(n, min = 0, max = 10)

# Using pipes for readability
if (!requireNamespace("dplyr", quietly = TRUE)) install.packages("dplyr")
library(dplyr)

tibble(x = x) %>%
  summarise(
    mean_x = mean(x),
    sd_x   = sd(x)
  )
```

```
## # A tibble: 1 x 2
##   mean_x  sd_x
##    <dbl> <dbl>
## 1   4.91  2.93
```

A2. Generate u and summarize (C8-ii)

Normal(0,36) means mean 0 and SD 6.

```r
u <- rnorm(n, mean = 0, sd = 6)

tibble(u = u) %>%
  summarise(
    mean_u = mean(u),
    sd_u   = sd(u)
  )
```

```
## # A tibble: 1 x 2
##    mean_u  sd_u
##     <dbl> <dbl>
## 1 -0.0174  6.23
```

Why is the sample mean of `u` not exactly zero?

- In the population, $\mathbf{E(u)=0}$.
- In a finite random sample, the sample mean is almost never exactly 0.
- As `n` grows, the sample mean tends to get closer to 0 (law of large numbers).

A3. Generate y and run OLS (C8-iii)

```r
y <- 1 + 2*x + u

fit <- lm(y ~ x)
summary(fit)
```

```
##
## Call:
## lm(formula = y ~ x)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -21.0262  -4.3694  -0.2483   4.6411  17.6327
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.6659     0.5450   1.222    0.222
## x             2.0645     0.0954  21.642   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.237 on 498 degrees of freedom
## Multiple R-squared:  0.4847, Adjusted R-squared:  0.4836
## F-statistic: 468.4 on 1 and 498 DF,  p-value: < 2.2e-16
```

```
coef(fit)
```

```
## (Intercept)           x
##   0.6658578   2.0645379
```

Interpretation:

- `b0_hat` estimates the true intercept (1).
- `b1_hat` estimates the true slope (2).
- They generally differ from (1,2) in any one sample because of sampling variation.

A4. Verify residual properties (C8-iv)

Equation (2.60) is the pair of sample identities:

- `sum(u_hat_i) = 0`
- `sum(x_i * u_hat_i) = 0`

(up to tiny rounding error).

```
uhat <- resid(fit)

c(
  sum_uhat   = sum(uhat),
  sum_x_uhat = sum(x * uhat)
)
```

```
##     sum_uhat   sum_x_uhat
## 2.210107e-13 1.072864e-12
```

A5. Replace residuals with true errors (C8-v)

Now compute the same quantities, but use `u_i`:

```r
c(
  sum_u   = sum(u),
  sum_x_u = sum(x * u)
)
```

```
##      sum_u    sum_x_u
##  -8.677757 233.275426
```

Conclusion:

- The "sum-to-zero" and "orthogonality to x" properties are **guaranteed for OLS residuals**.
- They are **not** guaranteed for the unobserved true errors `u_i` in a finite sample.

A6. Repeat with a new sample (C8-vi)

```r
set.seed(376)

x_new <- runif(n, 0, 10)
u_new <- rnorm(n, 0, 6)
y_new <- 1 + 2*x_new + u_new

fit_new <- lm(y_new ~ x_new)
coef(fit_new)
```

```
## (Intercept)       x_new
##    1.631173    1.967550
```

Why are the estimates different?

- Because `x` and `u` are newly simulated, the sample is different.
- OLS estimators are random variables: they change across samples.
- With larger `n`, the estimates cluster more tightly around (1,2).

Part B — Extension: Inference tools for OLS

C8 does not explicitly require inference, but the attached script demonstrates the standard workflow:

- Confidence intervals for coefficients
- Hypothesis tests for coefficients
- Confidence intervals for conditional means
- Prediction intervals for new outcomes

We demonstrate these using the simulated SLR model above.

B1. Confidence intervals

```r
# "Exact" t-based CI (assuming normal errors)
confint(fit)
```

```
##                   2.5 %   97.5 %
## (Intercept) -0.4050042 1.736720
## x            1.8771077 2.251968
```

```
# Large-sample (normal approximation)
confint.default(fit)
```

```
##                   2.5 %   97.5 %
## (Intercept) -0.4024017 1.734117
## x            1.8775632 2.251513
```

Interlude: Testing Linear Restrictions with `car::linearHypothesis()`

Before we run hypothesis tests like $H_0 : \beta_x = 2$, it helps to know what `linearHypothesis()` is doing and why it is useful.

What is `linearHypothesis()`?

`linearHypothesis()` (from the **car** package) is a convenient tool for testing **linear restrictions** on regression coefficients.

A **linear restriction** is any hypothesis that can be written as:

$$H_0 : \ R\beta = r,$$

where R is a matrix of constants and r is a vector of constants.

Common examples:

- **Single restriction** (one coefficient equals a number):

$$H_0 : \ \beta_x = 2$$

- **Joint restrictions** (multiple restrictions at once):

$$H_0 : \ \beta_{x1} = 0, \quad \beta_{x2} = 0$$

- **Linear combination** (sum/difference of coefficients):

$$H_0 : \ \beta_1 + \beta_2 = 0$$

Under the hood, `linearHypothesis()` performs a **Wald test** (often reported as an **F-test** for `lm` models).

Step-by-step: How to use it

```
if (!requireNamespace("car", quietly = TRUE)) install.packages("car")
library(car)
```

Step 0 — Install/load required packages

Step 1 — Fit a model In this tutorial, we will reuse the model object `fit` created earlier (from the simulated SLR example).

```r
# Confirm that 'fit' exists and inspect coefficient names
exists("fit")
```

```
## [1] TRUE
```

```r
names(coef(fit))
```

```
## [1] "(Intercept)" "x"
```

```r
coef(fit)
```

```
## (Intercept)           x
##   0.6658578   2.0645379
```

Step 2 — Write restrictions using coefficient names Restrictions must match coefficient names exactly (as shown by `names(coef(fit))`).

Step 3 — Test restrictions (A) Test a single restriction: $H_0 : \beta_x = 0$

```r
linearHypothesis(fit, "x = 0")
```

```
## Linear hypothesis test
##
## Hypothesis:
## x = 0
##
## Model 1: restricted model
## Model 2: y ~ x
##
##   Res.Df   RSS Df Sum of Sq      F    Pr(>F)
## 1    499 37592
## 2    498 19373  1     18220 468.36 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

(B) Test a single restriction: $H_0 : \beta_x = 2$

```r
linearHypothesis(fit, "x = 2")
```

```
## Linear hypothesis test
##
## Hypothesis:
## x = 2
##
## Model 1: restricted model
## Model 2: y ~ x
##
##   Res.Df   RSS Df Sum of Sq      F Pr(>F)
## 1    499 19391
## 2    498 19373  1    17.804 0.4577  0.499
```

How to interpret the output

For `lm` models, `linearHypothesis()` prints an ANOVA-style table with:

- **Df**: number of restrictions being tested (call it q)
- **F statistic** and **p-value**
- Reject H_0 if p-value $< \alpha$

Useful fact: If there is only **one restriction** ($q = 1$), the test is equivalent to a t-test:

$$F = t^2.$$

Base R alternative (nested-model F test)

Base R can test joint restrictions by comparing a restricted and unrestricted model:

```
fit_u <- lm(y ~ x)   # unrestricted
fit_r <- lm(y ~ 1)   # restricted: forces slope on x = 0
anova(fit_r, fit_u)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ 1
## Model 2: y ~ x
##   Res.Df   RSS Df Sum of Sq      F    Pr(>F)
## 1    499 37592
## 2    498 19373  1     18220 468.36 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

This works nicely when restrictions are "drop a regressor" restrictions.
For restrictions like $\beta_1 + \beta_2 = 0$, `linearHypothesis()` is usually simpler and more readable.

Robust inference (optional, applied work)

By default, `linearHypothesis()` uses the usual (homoskedastic) OLS covariance matrix.
For heteroskedasticity-robust inference, supply a robust `vcov.`:

```
# install.packages("sandwich")  # run once if needed
library(sandwich)
linearHypothesis(fit, "x = 2", vcov. = vcovHC(fit, type = "HC1"))
```

Common pitfalls

1. **Coefficient name mismatch:** always check `names(coef(fit))` before writing restrictions.
2. **Only linear restrictions:** it does not test nonlinear restrictions like $\beta_1\beta_2 = 1$.
3. **Defaults:** standard errors are homoskedastic unless you pass a robust `vcov.`.
4. **Identification issues:** perfect multicollinearity can make some restrictions non-estimable.

B2. Hypothesis tests (single restriction)

Test H0: slope = 0 or slope = 2.

```r
if (!requireNamespace("car", quietly = TRUE)) install.packages("car")
library(car)

linearHypothesis(fit, "x = 0")
```

```
## Linear hypothesis test
##
## Hypothesis:
## x = 0
##
## Model 1: restricted model
## Model 2: y ~ x
##
##   Res.Df   RSS Df Sum of Sq      F    Pr(>F)
## 1    499 37592
## 2    498 19373  1     18220 468.36 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
linearHypothesis(fit, "x = 2")
```

```
## Linear hypothesis test
##
## Hypothesis:
## x = 2
##
## Model 1: restricted model
## Model 2: y ~ x
##
##   Res.Df   RSS Df Sum of Sq      F Pr(>F)
## 1    499 19391
## 2    498 19373  1    17.804 0.4577  0.499
```

B3. Mean vs prediction intervals

```r
x0 <- c(0.1, 0.5, 0.9)

# Mean response E[Y|X=x0]
predict(fit, newdata = data.frame(x = x0), interval = "confidence")
```

```
##         fit        lwr      upr
## 1 0.8723116 -0.1824914 1.927115
## 2 1.6981267  0.7066174 2.689636
## 3 2.5239419  1.5939883 3.453896
```

```r
# New outcome Y|X=x0
predict(fit, newdata = data.frame(x = x0), interval = "prediction")
```

```
##         fit        lwr      upr
## 1 0.8723116 -11.427196 13.17182
## 2 1.6981267 -10.596114 13.99237
## 3 2.5239419  -9.765488 14.81337
```

Part C — Extension: Multiple regression and omitted variable bias (simulation)

The attached `tutorial_5.R` also simulates a case where `x2` is correlated with `x1`, and the true model is:

- `y = 1 + 1*x1 + 2*x2 + u`

Then it compares:

- SLR: `y ~ x1` (misspecified; omits x2)
- MLR: `y ~ x1 + x2` (correct)

C1. One draw: compare SLR vs MLR

```
set.seed(999)

n <- 300
x1 <- runif(n)
x2 <- 0.5 * exp(x1)^2 + runif(n)
u  <- rnorm(n)
y  <- 1 + 1*x1 + 2*x2 + u

dat <- tibble(y = y, x1 = x1, x2 = x2)

fit_slr <- lm(y ~ x1, data = dat)
fit_mlr <- lm(y ~ x1 + x2, data = dat)

summary(fit_slr)
```

```
##
## Call:
## lm(formula = y ~ x1, data = dat)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.5613 -0.8200 -0.0573  0.7799  3.4853
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   2.3377     0.1308   17.88   <2e-16 ***
## x1            6.6890     0.2327   28.74   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.158 on 298 degrees of freedom
## Multiple R-squared:  0.7349, Adjusted R-squared:  0.734
## F-statistic: 825.9 on 1 and 298 DF,  p-value: < 2.2e-16
```

```
summary(fit_mlr)
```

```
##
## Call:
```

```
## lm(formula = y ~ x1 + x2, data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.60837 -0.63289 -0.04435  0.63148  3.06539
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   1.1594     0.1478   7.842 8.02e-14 ***
## x1            1.3803     0.4926   2.802  0.00541 **
## x2            1.8370     0.1568  11.713  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9592 on 297 degrees of freedom
## Multiple R-squared:  0.8186, Adjusted R-squared:  0.8174
## F-statistic: 670.3 on 2 and 297 DF,  p-value: < 2.2e-16
```

C2. Monte Carlo: sampling distribution of estimates

```r
set.seed(1001)

B <- 1000
n <- 300

beta_slr <- matrix(NA, nrow = B, ncol = 2)  # (intercept, slope on x1)
beta_mlr <- matrix(NA, nrow = B, ncol = 3)  # (intercept, slope on x1, slope on x2)

for (b in 1:B) {
  x1 <- runif(n)
  x2 <- 0.5 * exp(x1)^2 + runif(n)
  u  <- rnorm(n)
  y  <- 1 + 1*x1 + 2*x2 + u

  beta_slr[b, ] <- coef(lm(y ~ x1))
  beta_mlr[b, ] <- coef(lm(y ~ x1 + x2))
}

colnames(beta_slr) <- c("b0_hat", "b1_hat_slr")
colnames(beta_mlr) <- c("b0_hat", "b1_hat_mlr", "b2_hat_mlr")

# Summaries (pipe-friendly)
tibble(beta_slr) %>% summarise(across(everything(), list(mean = mean, sd = sd)))
```

```
## # A tibble: 1 x 2
##   beta_slr_mean beta_slr_sd
##           <dbl>       <dbl>
## 1          4.59        2.40
```

```r
tibble(beta_mlr) %>% summarise(across(everything(), list(mean = mean, sd = sd)))
```

```
## # A tibble: 1 x 2
##   beta_mlr_mean beta_mlr_sd
##           <dbl>       <dbl>
## 1          1.34       0.569
```

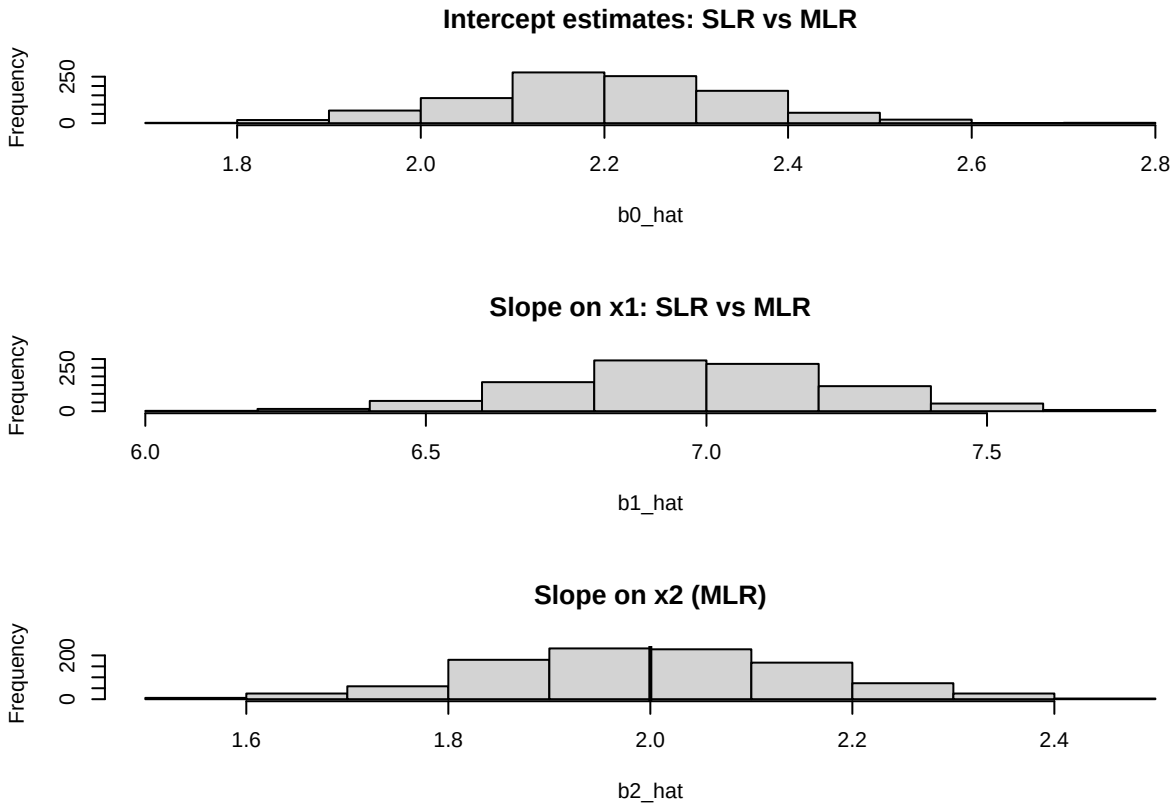
Optional: quick histograms

```r
par(mfrow = c(3, 1))

hist(beta_slr[, 1], main = "Intercept estimates: SLR vs MLR", xlab = "b0_hat")
hist(beta_mlr[, 1], add = TRUE)
abline(v = 1, lwd = 2)  # true intercept

hist(beta_slr[, 2], main = "Slope on x1: SLR vs MLR", xlab = "b1_hat")
hist(beta_mlr[, 2], add = TRUE)
abline(v = 1, lwd = 2)  # true b1

hist(beta_mlr[, 3], main = "Slope on x2 (MLR)", xlab = "b2_hat")
abline(v = 2, lwd = 2)  # true b2
```



Wrap-up

- **C8 (Chapter 2)** is about simulation and the algebraic properties of OLS residuals.
- Then adds **inference** tools and a **multiple-regression simulation** to build intuition.
- The key mental model: OLS outputs are *functions of the sample*, so they vary across samples.